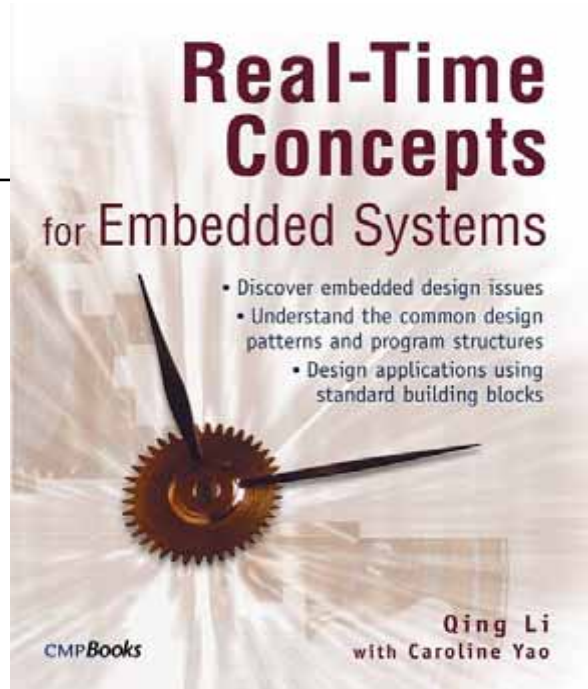
Real-Time Concepts for Embedded Systems

Author: Qing Li with Caroline Yao

ISBN: 1-57820-124-1

CMP*Books*



Chapter 7
Message Queues

Outline

- 7.1 Introduction
- 7.2 Defining Message Queues
- 7.3 Message Queue States
- 7.4 Message Queue Content
- 7.5 Message Queue Storage
- 7.6 Typical Message Queue Operations
- 7.7 Typical Message Queue Use

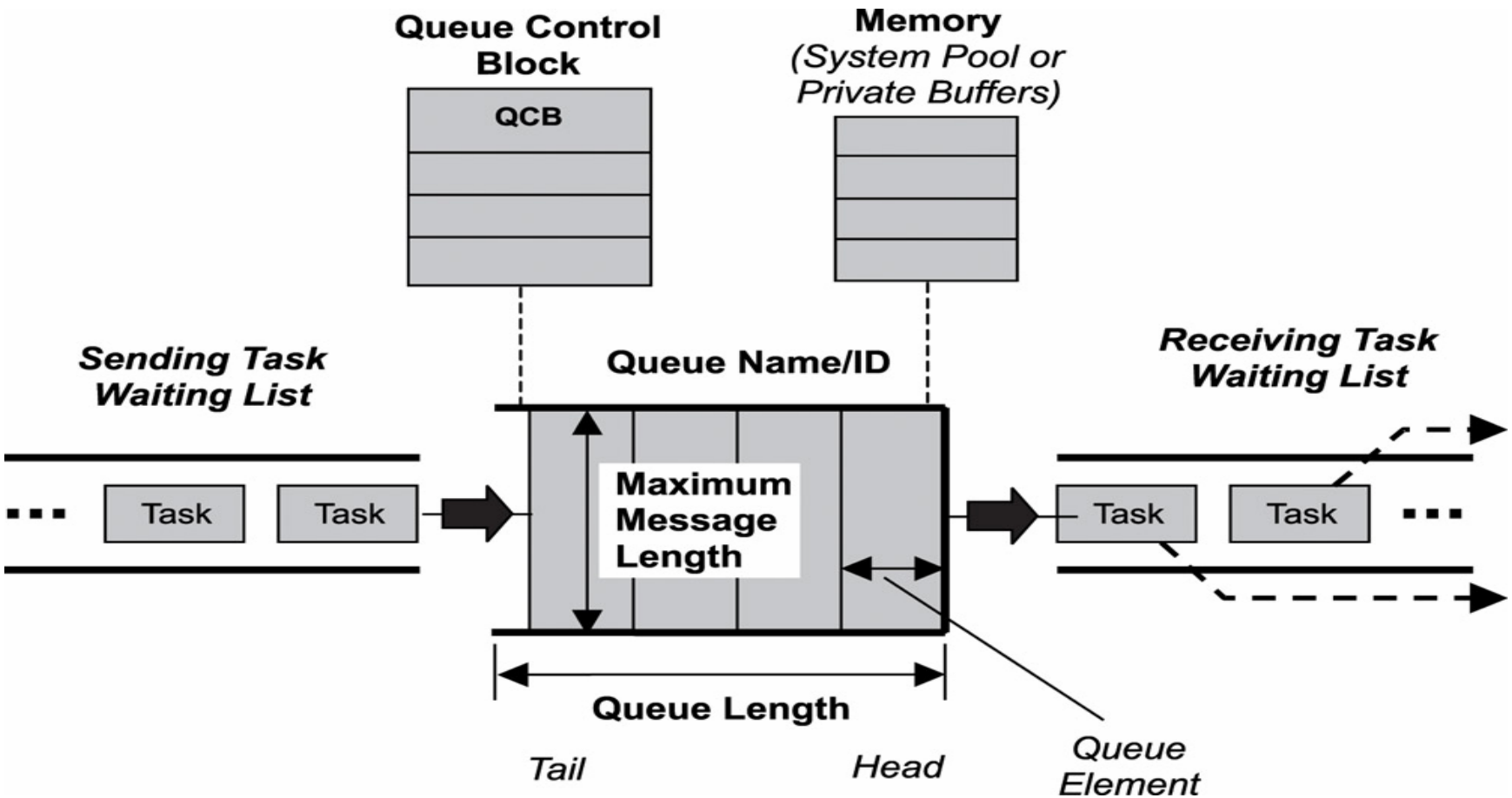
7.1 Introduction

- To facilitate inter-task data communication, kernels provide
  - a message queue object and message queue management services

7.2 Defining Semaphores

- A *message queue*
  - a buffer-like object through which tasks and ISRs send and receive messages to communicate and synchronize with data
- When a message queue is first created, it is assigned
  - a queue control block (QCB)
  - a message queue name
  - a unique ID
  - memory buffers
  - a queue length
  - a maximum message length
  - task-waiting lists

A message queue, its associated parameters, and supporting data structures

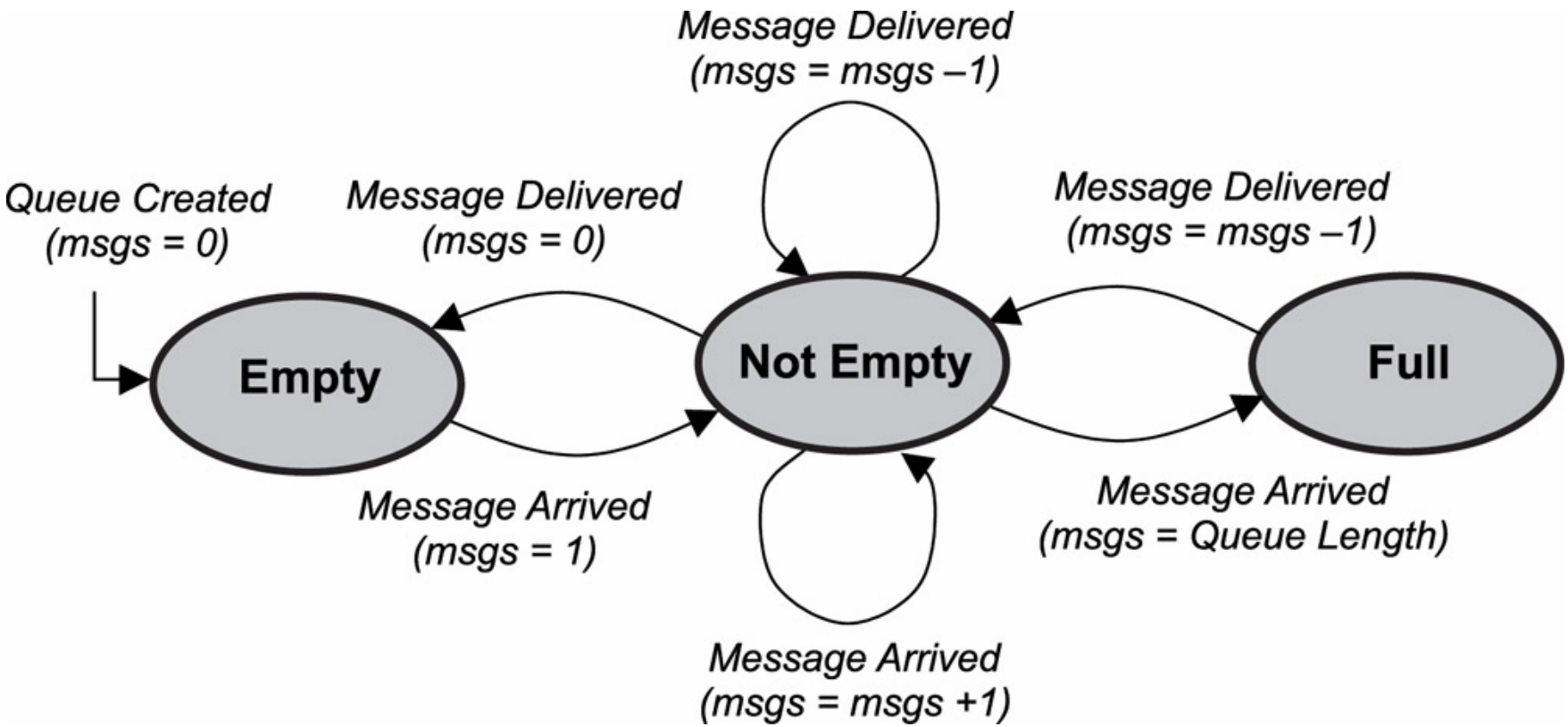


Message Queue

- The message queue itself consists of a number of elements, each of which can hold a single message.
- Kernel takes developer-supplied parameters to determine how much memory is required for the message queue:
  - queue length and maximum message length

7.3 Message Queue States

- The state diagram for a message queue:



Message Queue States (Cont.)

- When a task attempts to send a message to a full message queue, two ways of kernel implementation:
  - the sending function returns an error code to that task
  - Sending task is blocked and is moved into sending task-waiting list

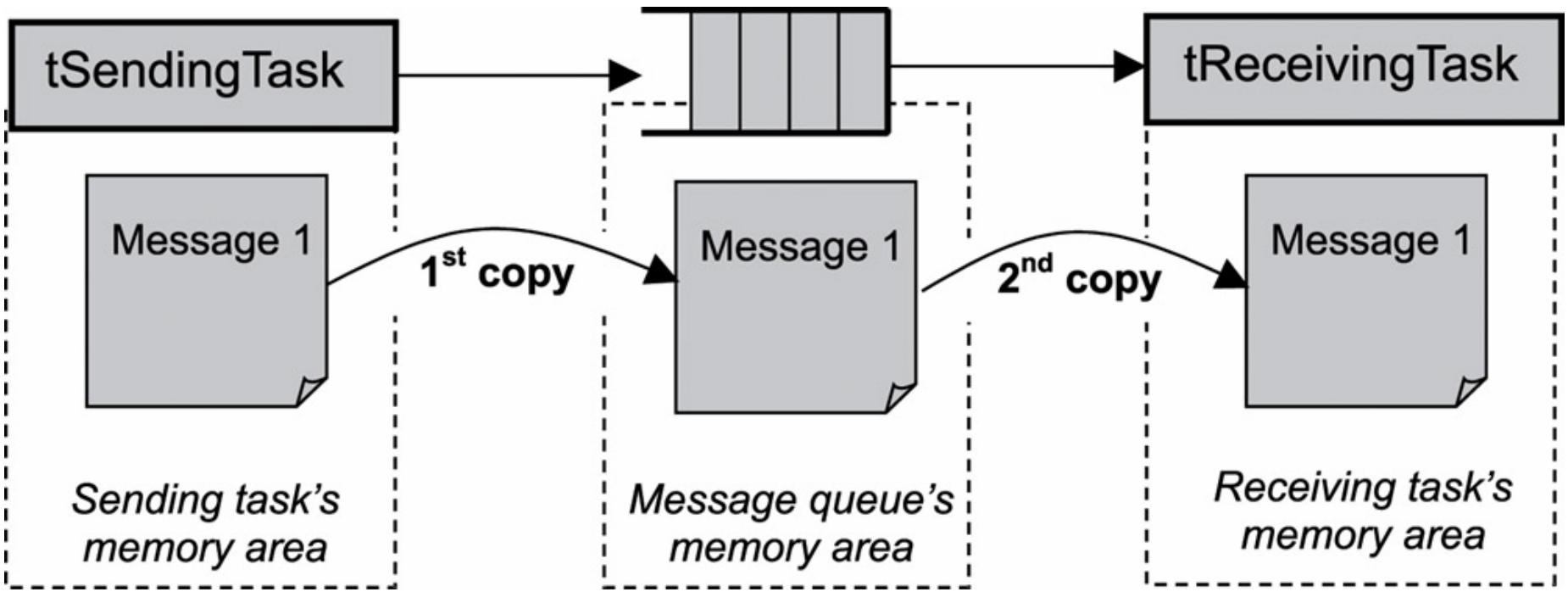
7.4 Message Queue Content

- Message queues can be used to send and receive a variety of data. Some examples:
  - a temperature value from a sensor
  - a bitmap to draw on a display
  - a text message to print to an LCD
  - a keyboard event
  - a data packet to send over the network

Message Queue Content (Cont.)

- When a task sends a message to another task, the message normally is copied twice
  - from sender's memory area to the message queue's memory area
  - from the message queue's memory area to receiver's memory area
- Copying data can be expensive in terms of performance and memory requirements

Message copying and memory use for sending and receiving messages



Message Queue Content (Cont.)

- Keep copying to a minimum in a real-time embedded system:
  - by keeping messages small
  - by using a pointer instead
- Send a pointer to the data, rather than the data itself
  - overcome the limit on message length
  - improve both performance and memory utilization

7.5 Message Queue Storage

- Message queues may be stored in a system pool or private buffers
- System Pools
  - the messages of all queues are stored in one large shared area of memory
  - Advantage: save on memory use
  - Downside: a message queue with large messages can easily use most of the pooled memory

Message Queue Storage (Cont.)

- Private Buffers
  - separate memory areas for each message queue
  - Downside: uses up more memory
    - requires enough reserved memory area for the full capacity of every message queue that will be created
  - Advantage: better reliability
    - ensures that messages do not get overwritten and that room is available for all messages

7.6 Typical Message Queue Operations

- creating and deleting message queues
- sending and receiving messages
- obtaining message queue information

7.6.1 Creating and Deleting Message Queues

- When created, message queues are treated as global objects and are not owned by any particular task.
- When creating a message queue, a developer needs to decide
  - message queue length
  - the maximum messages size
  - the blocked tasks waiting order

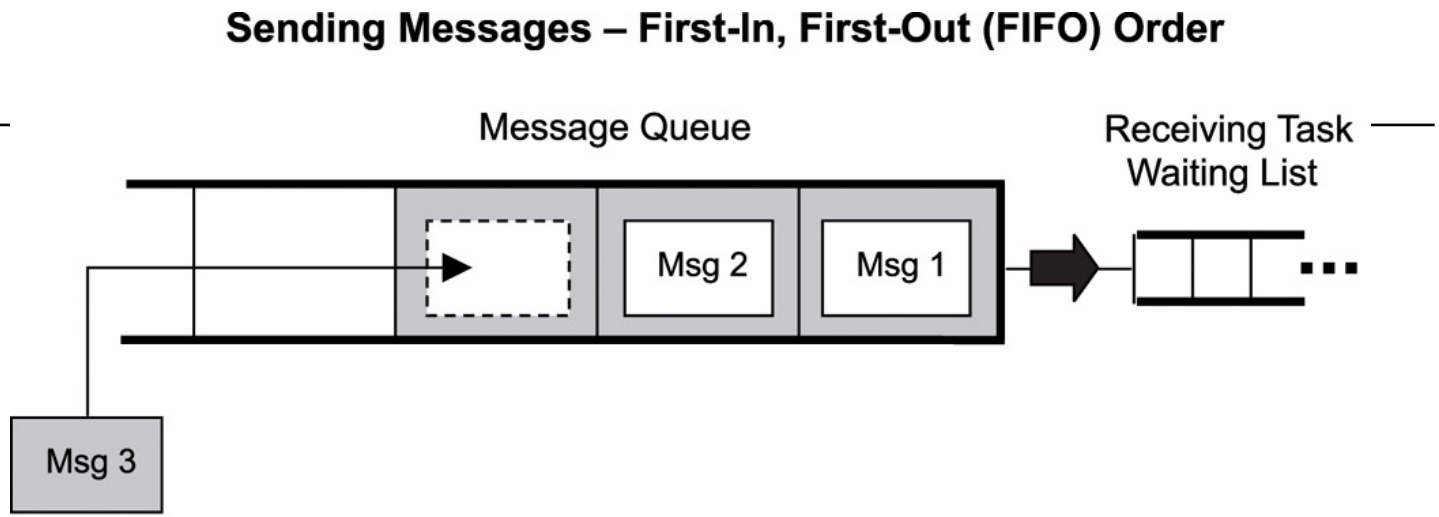
Message queue creation and deletion operations

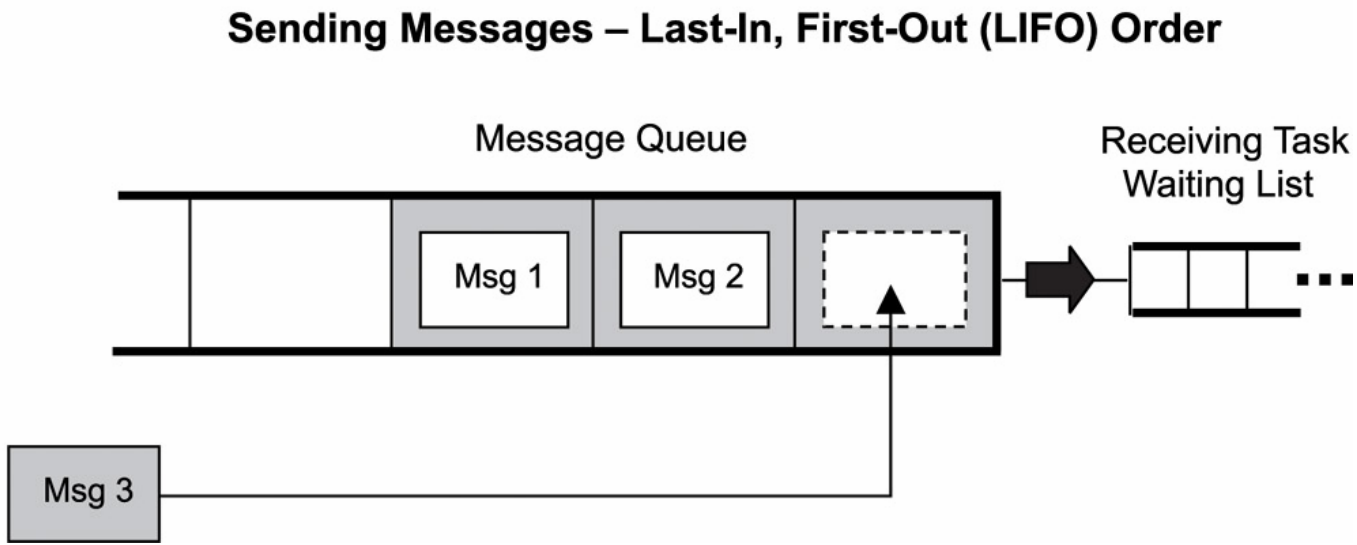
Operation	Description
Create	Creates a message queue
Delete	Deletes a message queue

7.6.2 Sending and Receiving Messages

Operation	Description
Send	Sends a message to a message queue
Receive	Receives a message from a message queue
Broadcast	Broadcasts messages

Sending messages in FIFO or LIFO order

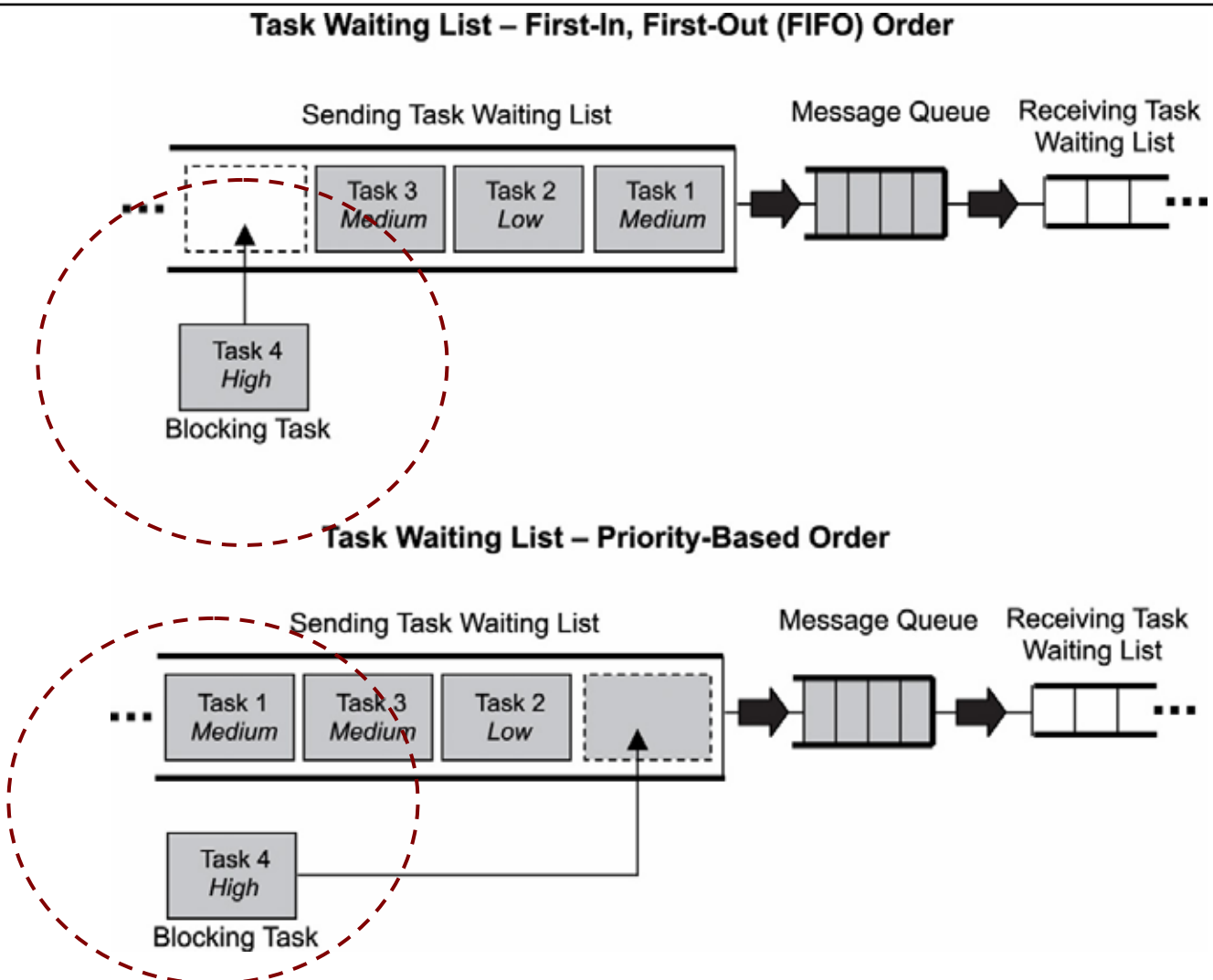




Sending Messages

- Tasks can send messages with different blocking policies:
  - not block (ISRs and tasks)
    - If a message queue is already full, the send call returns with an error, the sender does not block
  - block with a timeout (tasks only)
  - block forever (tasks only)
- The blocked task is placed in the message queue's task-waiting list
  - FIFO or priority-based order

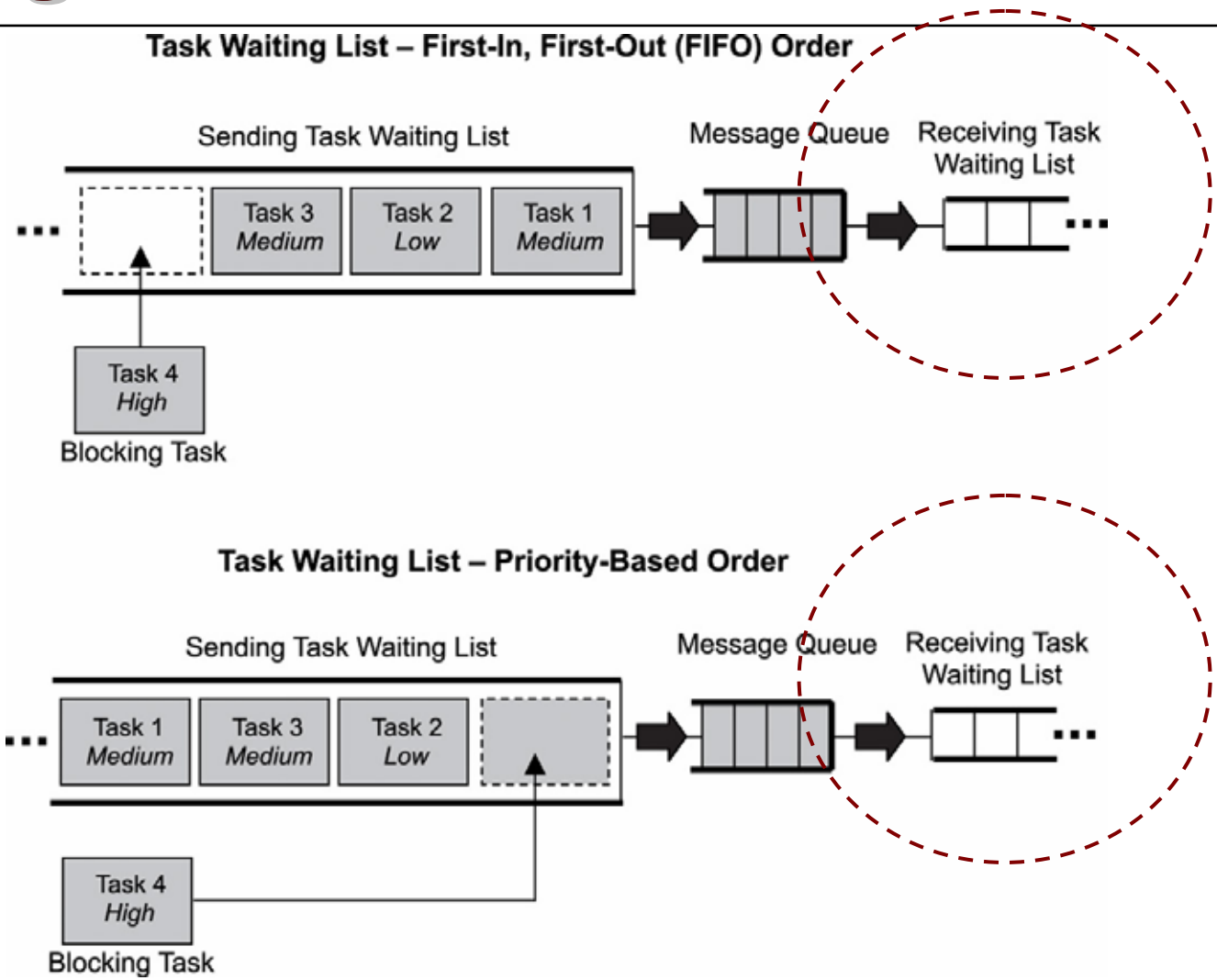
FIFO and priority-based task-waiting lists



Receiving Messages

- Tasks can receive messages with different blocking policies:
  - not blocking
  - blocking with a timeout
  - blocking forever
- Due to the empty message queue, the blocked task is placed in the message queue's task-waiting list
  - FIFO or priority-based order

FIFO and priority-based task-waiting lists



Receiving Messages (Cont.)

- Messages can be read from the head of a message queue in two different ways:
  - destructive read
    - removes the message from the message queue's storage buffer after successfully read
  - non-destructive read
    - without removing the message

7.6.3 Obtaining Message Queue Information

- Obtain information about a message queue:
  - message queue ID, task-waiting list queuing order (FIFO or priority-based), and the number of messages queued.

Operation	Description
Show queue info	Gets information on a message queue
Show queue's task-waiting list	Gets a list of tasks in the queue's task-waiting list

7.7 Typical Message Queue Use

- Typical ways to use message queues within an application:
  - non-interlocked, one-way data communication
  - interlocked, one-way data communication
  - interlocked, two-way data communication
  - broadcast communication

7.7.1 Non-Interlocked, One-Way Data Communication

- Non-interlocked (or loosely coupled), one-way data communication:
  - The activities of tSourceTask and tSinkTask are not synchronized.
    - TSourceTask simply sends a message and does not require acknowledgement from tSinkTask.

Non-Interlocked, One-Way Data Communication

```
tSourceTask ()
{
        :
        Send message to message queue
        :
}

tSinkTask ()
{
        :
        Receive message from message queue
        :
}
```

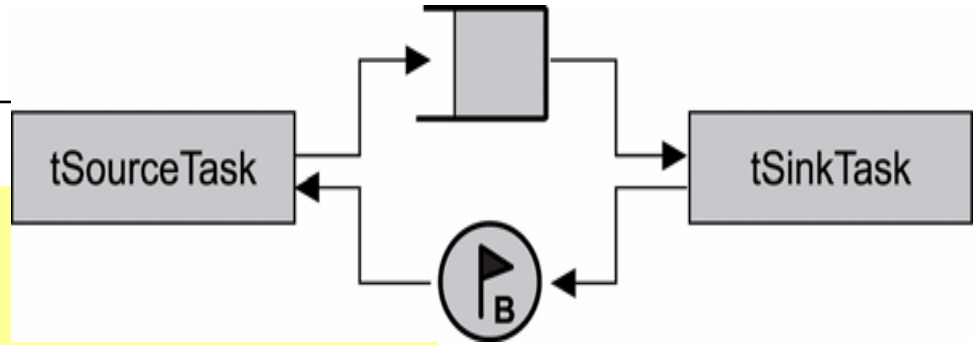
Non-Interlocked, One-Way Data Communication (Cont.)

- ISRs typically use non-interlocked, one-way communication.
  - A task such as tSinkTask runs and waits on the message queue.
  - When the hardware triggers an ISR to run, the ISR puts one or more messages into the message queue for tSinkTask.
- ISRs send messages to the message queue in a non-blocking way.
  - If the message queue becomes full, any additional messages that the ISR sends to the message queue are lost.

7.7.2 Interlocked, One-Way Data Communication

- Interlocked communication
  - the sending task sends a message and waits to see if the message is received
  - useful for reliable communications or task synchronization
- Solution
  - a binary semaphore initially set to 0 and a message queue with a length of 1 (also called a mailbox)
  - Sender *tSourceTask* and receiver *tSinkTask* operate in lockstep with each other

Interlocked, One-Way Data Communication



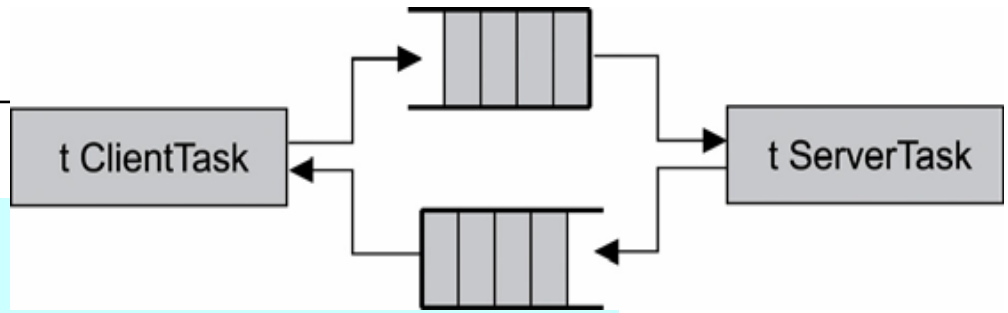

```
tSourceTask ()
{
        :
        Send message to message queue
        Acquire binary semaphore
        :
}

tSinkTask ()
{
        :
        Receive message from message queue
        Give binary semaphore
        :
}
```

7.7.3 Interlocked, Two-Way Data Communication

- Interlocked, two-way data communication (also called full-duplex or tightly coupled communication)
  - data flow bidirectionally between tasks
  - useful when designing a client/server-based system
  - *two separate message queues are required*
- If multiple clients need to be set up
  - all clients can use the client message queue to post requests
  - tServerTask uses a separate message queue to fulfill the different clients' requests

Interlocked, Two-Way Data Communication



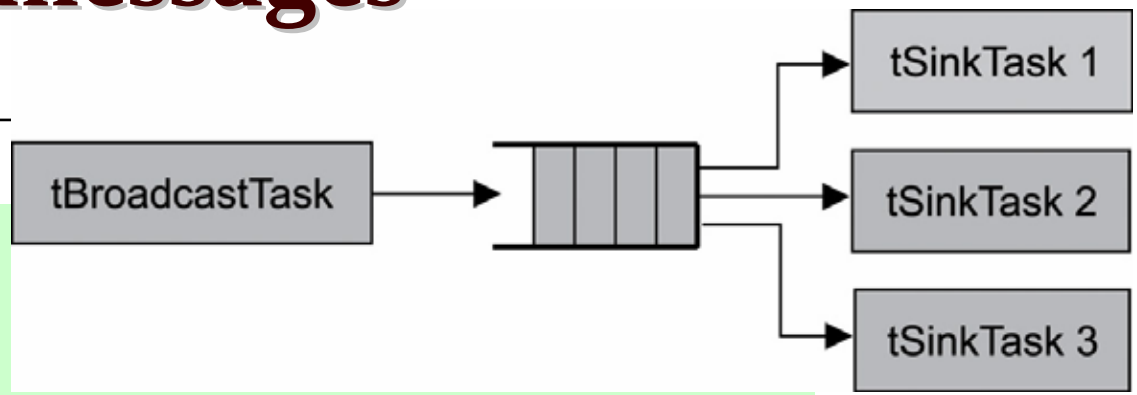

```
tClientTask ()
{
        :
        Send a message to the requests queue
        Wait for message from the server queue
        :
}

tServerTask ()
{
        :
        Receive a message from the requests queue
        Send a message to the client queue
        :
}
```

7.7.4 Broadcast Communication

- Allow developers to broadcast a copy of the same message to multiple tasks
- Message broadcasting is a one-to-many-task relationship.
  - tBroadcastTask sends the message on which multiple tSink-Task are waiting.

Broadcasting messages




```
tBroadcastTask ()
{
        :
        Send broadcast message to queue
        :
}
```

Note: similar code for tSignalTasks 1, 2, and 3.

```
tSignalTask ()
{
        :
        Receive message on queue
        :
}
```